Boolean Equivalence and Finite-State Models

A confident claim still needs every check

COMSC 2043 ▪ Discrete Structures and Critical Thinking

Week 14, Tuesday, November 10, 2026

Today's question

PRINCIPLE: EQUIVALENCE IS A UNIVERSAL CLAIM

The question

When may we say that two Boolean expressions mean the same thing?

- They must produce the same output for every allowed input.
- A single different output is enough to reject the claim.
- We will make each check visible before asking Claude for code.

PREDICT (30 s): What evidence would establish same for every input?

A smoke alarm for a strong claim

- A smoke alarm test can find one bad room.
- It cannot certify that every room is safe unless every room was checked.
- A counterexample is one allowed case that breaks a universal claim.

Our loop

Predict → run → compare → explain

What you should see: a checkable row, not a confident sentence.

The confident AI claim

Original expression

$F = (P \text{ OR } Q) \text{ AND } (P \text{ OR NOT } Q)$

AI simplification

$B = P \text{ OR } Q$

- The claim is: F and B are equivalent.
- Equivalent means equal for every assignment, not merely for one example.

PREDICT (30 s): Which values of P, Q would you test first? Why?

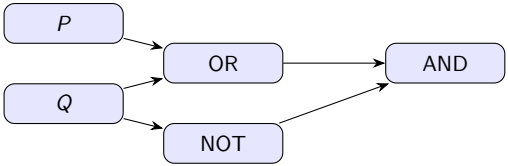
Words before symbols

Boolean variable	A named value that is either True or False, such as P.
Assignment	The values chosen for the variables, such as $P = \mathit{False}, Q = \mathit{True}$.
AND	True only when both inputs are True.
OR	True when at least one input is True.
NOT	Flips True to False, or False to True.
Expression	A rule built from variables and Boolean operations.
Equivalence	Two expressions have equal outputs for every assignment.

From expression to circuit

A circuit

A circuit is a picture of gates connected by wires. A gate applies one Boolean operation to its inputs.



- OR gate: $P \vee Q$
- NOT gate: $\neg Q$
- AND gate: combine the two results

What you should see: the circuit and the expression describe the same calculation.

A truth table checks all assignments

PRINCIPLE: EXHAUSTIVE CHECK

P	Q	$P \vee Q$	$P \vee \neg Q$	F
False	False	False	True	False
False	True	True	False	False
True	False	True	True	True
True	True	True	True	True

Truth table: a table that lists an expression's output for every assignment.
What you should see: the second row already makes the AI claim suspicious.

One row defeats the universal claim

PRINCIPLE: COUNTEREXAMPLE

P	Q	F	$B = P \vee Q$
False	False	False	False
False	True	False	True
True	False	True	True
True	True	True	True

Counterexample

$P = \mathit{False}, Q = \mathit{True}$ gives different outputs. Therefore F and B are not equivalent.

Common mistake: Three matching rows do not cancel one mismatch.

What counts as proof here?

PRINCIPLE: SOURCES, RULES, WORK, CHECK

1. State the allowed variables and operations.
2. List every assignment.
3. Compute both outputs on every row.
4. Name any differing row.

Universal claim

Same for every assignment is supported only when every assignment passes, or when a valid general proof covers them all.

PREDICT (30 s): What would a program need to print so you could audit it?

The correct simplification from the activity

PRINCIPLE: COMPARE CLAIMS

Source expression

$E = (P \text{ AND } Q) \text{ OR } (P \text{ AND NOT } Q)$

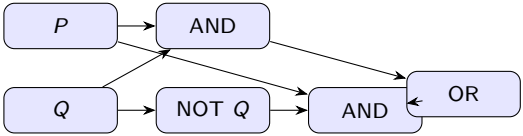
- In both terms, P must be True.
- If P is True, either Q or NOT Q is True.
- Therefore $E = P$ for every assignment.

What you should see: the source activity's correct result is P, not P OR Q.

Expression and circuit agree

Expression

$E = (P \wedge Q) \vee (P \wedge \neg Q)$



- two AND gates
- one NOT gate
- one OR gate

PREDICT (30 s): Which circuit output should match P on all four rows?

Prompt 01: small steps first

PRINCIPLE: PREDICT, RUN, EXPLAIN

PROMPT FOR CLAUDE

prompts/01_session_rules.md

```
Act as a careful checking partner for a two-variable Boolean example. Before writing code, state what an
  expression, a Boolean variable, an assignment, and equivalence mean in plain language. Use only the operators
  AND, OR, and NOT. Keep the example synthetic and self-contained. Then propose the smallest Python standard-
  library evaluator you would need for a truth table. Do not claim two expressions are equivalent until every
  assignment has been checked. Show one predicted row before showing the rest.
```

What you should see: one visible Boolean result, plus a visible failure for a missing variable if you try it.

PREDICT (30 s): What should P AND NOT Q return for $P = \mathit{True}$, $Q = \mathit{False}$?

Prompt 02: enumerate every assignment

PRINCIPLE: CHECK THE WORK

PROMPT FOR CLAUDE

prompts/02_truth_table_counterexample.md

```
Write a small Python program using only the standard library. Define F = (P OR Q) AND (P OR NOT Q) and the
  proposed simplification B = P OR Q. Evaluate both expressions for all four Boolean assignments of P and Q, in
  the fixed order (False, False), (False, True), (True, False), (True, True). Print a compact table with P, Q, F,
  and B, then print the exact assignments where they differ. Do not simplify the expressions by inspection. The
  expected check is one mismatch, at P=False and Q=True.
```

What you should see: four rows in fixed order; the row $P = \mathit{False}, Q = \mathit{True}$ is the mismatch already visible on the slide.

Common mistake: A single test row cannot establish equivalence.

Prompt 03a: require the exact counterexample

PRINCIPLE: FAILURE MUST BE SPECIFIC

PROMPT FOR CLAUDE

prompts/03_equivalence_sat.md

```
Extend a small standard-library Boolean evaluator with an equivalence checker. It must enumerate every assignment
  for the variables that occur and return a list of counterexamples, not just True or False. Check that E = (P AND
   Q) OR (P AND NOT Q) is equivalent to S = P, and check that F = (P OR Q) AND (P OR NOT Q) is not equivalent to B
   = P OR Q. Print the counterexample for the second pair exactly as a mapping from variable names to Boolean
  values. State why an empty counterexample list supports equivalence only for the variables and operators
  actually supplied.
```

What you should see: equivalence = False and counterexample = `"P": False, "Q": True`.

PREDICT (30 s): Why is returning only False weaker than returning the assignment?

Three ways a check can speak

Output	Meaning	Example
matching rows	every listed assignment agrees	E matches P on all four rows
counterexample	one assignment disagrees	F vs B at False, True
witness	one assignment makes an expression True	P has witness $P = \mathit{True}$

What you should see: the shape of the evidence must match the strength of the claim.

Satisfiable means true somewhere

Satisfiable

An expression is satisfiable if at least one assignment makes it True.

Witness

A witness is one assignment that makes the expression True.

- P is satisfiable: witness $P = \mathit{True}$.
- That does not say P is True when $P = \mathit{False}$.

PREDICT (30 s): Is one witness enough to say an expression is always True?

Tautology means true everywhere

Tautology

A tautology is an expression that is True for every assignment.

expression	False, False	False, True / True, False / True, True
P OR NOT P	True	True
P AND NOT P	False	False

- A tautology has no counterexample.
- An unsatisfiable expression has no witness.

Prompt 03b: do not overclaim from a witness

PRINCIPLE: NAME THE LIMIT

PROMPT FOR CLAUDE

prompts/03_equivalence_sat.md

```
Using the same evaluator, write three separate checks. First, return one satisfying witness for P, meaning one
  assignment that makes P true. Second, decide whether P is a tautology, meaning true for every assignment, and if
   it is not, print one counterexample. Third, classify P OR NOT P and P AND NOT P as tautology or unsatisfiable.
  Explain in one sentence why a satisfying witness for P does not prove that P is always true. Do not use the
  words equivalent or always without showing the assignments that justify them.
```

What you should see: one witness for P, a counterexample to P being tautological, no counterexample for P OR NOT P, and no witness for P AND NOT P.

Common mistake: I found one True row does not mean all rows are True.

Boolean checks have a boundary

PRINCIPLE: MODEL LIMIT

What the check establishes

- outputs for the listed operations
- every listed assignment
- the named counterexample or witness

What it does not establish

- that an unlisted operation behaves correctly
- that a different input model has the same meaning
- that a real device follows the expression

What you should see: a conclusion is bounded by its variables, operations, and assignments.

From Boolean outputs to states

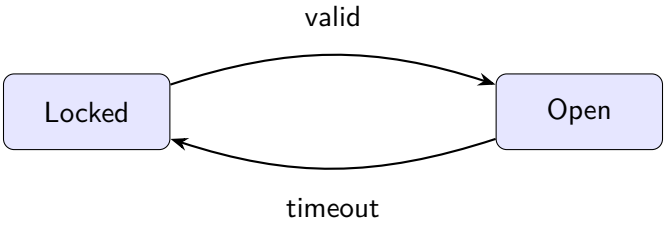
Finite-state machine

A finite-state machine (FSM) is a model with a finite list of states and rules for how named inputs move between them.

State	A named condition the model can be in, such as Locked.
Input	A named event the model receives, such as valid.
Transition	A rule mapping a current state and input to a next state.
Start state	The state where a run begins.
Trace	The start state and the sequence of transitions taken.

A complete two-state badge model

PRINCIPLE: EVERY TRANSITION IS EXPLICIT



- Start state: Locked.
- Declared inputs: valid and timeout.
- This is a synthetic teaching model, not a real access-control claim.

The supplied trace

$$\text{Locked} \xrightarrow{\text{valid}} \text{Open} \xrightarrow{\text{timeout}} \text{Locked} \xrightarrow{\text{valid}} \text{Open}$$

step	current	input	next
start	Locked	–	Locked
1	Locked	valid	Open
2	Open	timeout	Locked
3	Locked	valid	Open

What you should see: the trace includes the start state and every next state.

Prompt 04: print the trace

PROMPT FOR CLAUDE

prompts/04_fsm_trace.md

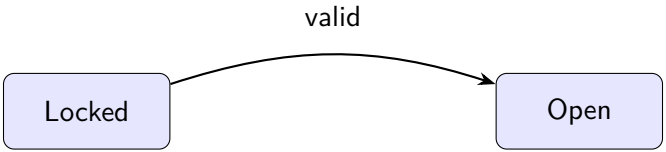
```
Implement a tiny finite-state-machine runner in Python with no third-party packages. The machine has states Locked
   and Open, inputs valid and timeout, start state Locked, and these transitions: (Locked, valid) -> Open and (
  Open, timeout) -> Locked. For this classroom trace, every other input leaves the state unchanged. Run the input
  sequence valid, timeout, valid and print the start state plus each resulting state. Also print a visible error
  for an input that is not in the declared alphabet. The expected state sequence is Locked, Open, Locked, Open. Do
   not invent authorization facts or physical-reader behavior.
```

What you should see: Locked, Open, Locked, Open, with the three inputs visible between them.

PREDICT (30 s): What is the state after valid, timeout, valid?

An incomplete model can look plausible

PRINCIPLE: MISSING RULES ARE EVIDENCE



- States: Locked, Open. Inputs: valid, timeout.
- Present rule: Locked + valid $\rightarrow$ Open.
- Missing rule: Open + timeout $\rightarrow$ Locked.

Common mistake: Do not invent a self-loop just because no arrow is drawn.

Prompt 05: name the exact gap

PROMPT FOR CLAUDE

prompts/05_model_limit.md

```
Write a validator for a finite-state machine whose declared states are Locked and Open and whose input alphabet is
   valid and timeout. The intentionally incomplete transition table contains only (Locked, valid) -> Open. List
  every missing (state, input) pair in deterministic order. The validator must identify exactly (Open, timeout) as
   the missing transition needed by the badge trace. Do not assume an implicit self-loop counts as a declared
  transition. Explain in one sentence what this validator can establish and what it cannot establish about a real
  access system.
```

What you should see: exactly one missing pair: (Open, timeout), plus a named model limit.

PREDICT (30 s): Which state/input pair is absent?

Tuesday activity: defend one claim

PRINCIPLE: INDIVIDUAL CHECK

1. Choose one expression pair or one small state trace.
2. Write the rule, variables/states, and expected result before using Claude.
3. Run one prompt and compare its visible output with your hand check.
4. Mark one match and one possible drift.

Peer and AI-fluency check

Ask your peer	Which row, witness, or transition supports your sentence?
Ask Claude	What assumption did you make, and what would make your answer fail?
Revise	Narrow the claim if the model checks less than you first said.

Good evidence

It names the input, the output, and the boundary of the model.

What a state model cannot establish

PRINCIPLE: MODEL LIMIT

- The badge model does not establish that a person is authorized.
- It does not establish the physical reader's condition.
- It does not cover an unlisted state such as Denied or Expired.
- It only checks the states, inputs, and transitions we declared.

Canonical Checkpoint 3 source

Use the linked source: `week-14/student/odyssey-checkpoint-3.md`. This deck does not copy or rewrite it.

Exit sentence

Complete this sentence

For these declared variables/states and inputs, my check shows _____ because _____; it does not establish _____.

- Include one row, witness, counterexample, or transition.
- Name one assumption or model limit.

What you should see: a conclusion that is true for the checked model, not for every possible system.

Index 1 of 4: principle, slide, prompt

Slide	Title	Principle	Prompt
2	Today's question	EQUIVALENCE IS A UNIVERSAL CLAIM	
3	A smoke alarm for a strong claim	CHECK THE WORK	
4	The confident AI claim	PREDICT BEFORE RUNNING	
5	Words before symbols	DEFINE THE MODEL	
6	From expression to circuit	MODEL VS PROCEDURE	
7	A truth table checks all assignments	EXHAUSTIVE CHECK	
8	One row defeats the universal claim	COUNTEREXAMPLE	
9	What counts as proof here?	SOURCES, RULES, WORK, CHECK	
10	The correct simplification from the activity	COMPARE CLAIMS	

Index 2 of 4: principle, slide, prompt

Slide	Title	Principle	Prompt
11	Expression and circuit agree	MODEL VS PROCEDURE	
12	Prompt 01: small steps first	PREDICT, RUN, EXPLAIN	01a
13	Prompt 02: enumerate every assignment	CHECK THE WORK	02a
14	Prompt 03a: require the exact counterexample	FAILURE MUST BE SPECIFIC	03a
15	Three ways a check can speak	EVIDENCE HAS A SHAPE	
16	Satisfiable means true somewhere	WITNESS VS TAUTOLOGY	
17	Tautology means true everywhere	UNIVERSAL CHECK	
18	Prompt 03b: do not overclaim from a witness	NAME THE LIMIT	03b
19	Boolean checks have a boundary	MODEL LIMIT	

Index 3 of 4: principle, slide, prompt

Slide	Title	Principle	Prompt
20	From Boolean outputs to states	MODEL THE NEXT STEP	
21	A complete two-state badge model	EVERY TRANSITION IS EXPLICIT	
22	The supplied trace	TRACE THE EVIDENCE	
23	Prompt 04: print the trace	PREDICT, RUN, EXPLAIN	04a
24	An incomplete model can look plausible	MISSING RULES ARE EVIDENCE	
25	Prompt 05: name the exact gap	VALIDATE THE MODEL	05a
26	Tuesday activity: defend one claim	INDIVIDUAL CHECK	
27	Peer and AI-fluency check	CRITIQUE THE EVIDENCE	
28	What a state model cannot establish	MODEL LIMIT	

Index 4 of 4: principle, slide, prompt

Slide	Title	Principle	Prompt
29	Exit sentence	BOUNDED CONCLUSION	